

ggplot2 and the Grammar of Graphics

- Follows a systematic **framework** for building visualizations by layering different components of a plot
- Graphs are composed of several key elements that can be combined and modified **independently**

Main Components

Component	Purpose	Example
Data (data)	The dataset you are visualizing.	ggplot(data = mtcars)
Aesthetics (aes())	Defines how data is mapped to visual properties (position, color, size).	aes(x = weight, y = height, color = gender) # This maps the weight and height variables to the x- and y-axes and assigns color based on gender.
Geometries (geom_*)	The type of plot or shape used to represent the data (points, lines, bars).	geom_point() # Scatter plot geom_line() # Line graph

```
library(ggplot2)
```

```
# Minimum required components  
ggplot(data = mtcars, aes(x = wt, y = mpg)) +  
  geom_point()
```

Optional Components

Component	Purpose	Example
Statistics (stat_*)	Statistical transformations, like computing counts or means.	stat_smooth(method = "lm") # Adds a linear regression line
Scales (scale_*)	Controls how data is mapped to aesthetic values (e.g., x/y scales, colors).	scale_x_continuous(limits = c(0, 100))
Facets (facet_*)	Splits the data into subplots based on categories.	facet_wrap(~gender)
Coordinates (coord_*)	Controls the coordinate system (e.g., Cartesian, polar).	coord_flip() # Flips x and y axes
Themes (theme())	Controls the non-data-related appearance (background, font size, grid lines).	theme_minimal()

Example

```
library(ggplot2)
```

```
# Example using Grammar of Graphics
```

```
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl))) + # Data + Aesthetics
```

```
  geom_point(size = 3) + # Geometry (scatter plot)
```

```
  geom_smooth(method = "lm", se = FALSE) + # Statistical transformation (linear model)
```

```
  scale_x_continuous(name = "Weight (1000 lbs)") + # Scale for x-axis
```

```
  scale_y_continuous(name = "Miles per Gallon") + # Scale for y-axis
```

```
  facet_wrap(~cyl) + # Faceting by number of cylinders
```

```
  theme_classic() # Theme for APA-style visualization
```

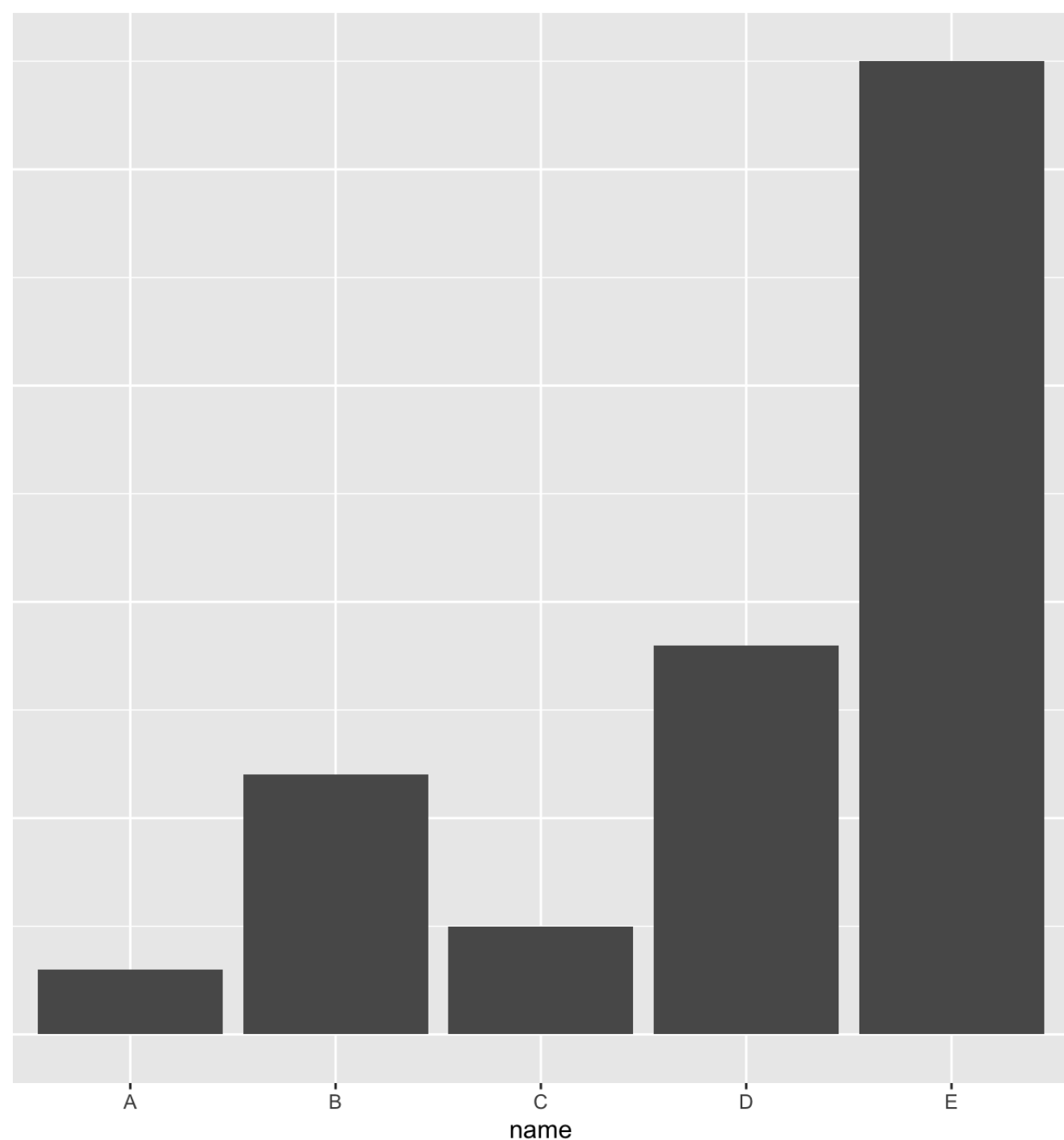
TYPES OF FIGURES

<https://r-graph-gallery.com/>

Bar Charts

geom_bar()

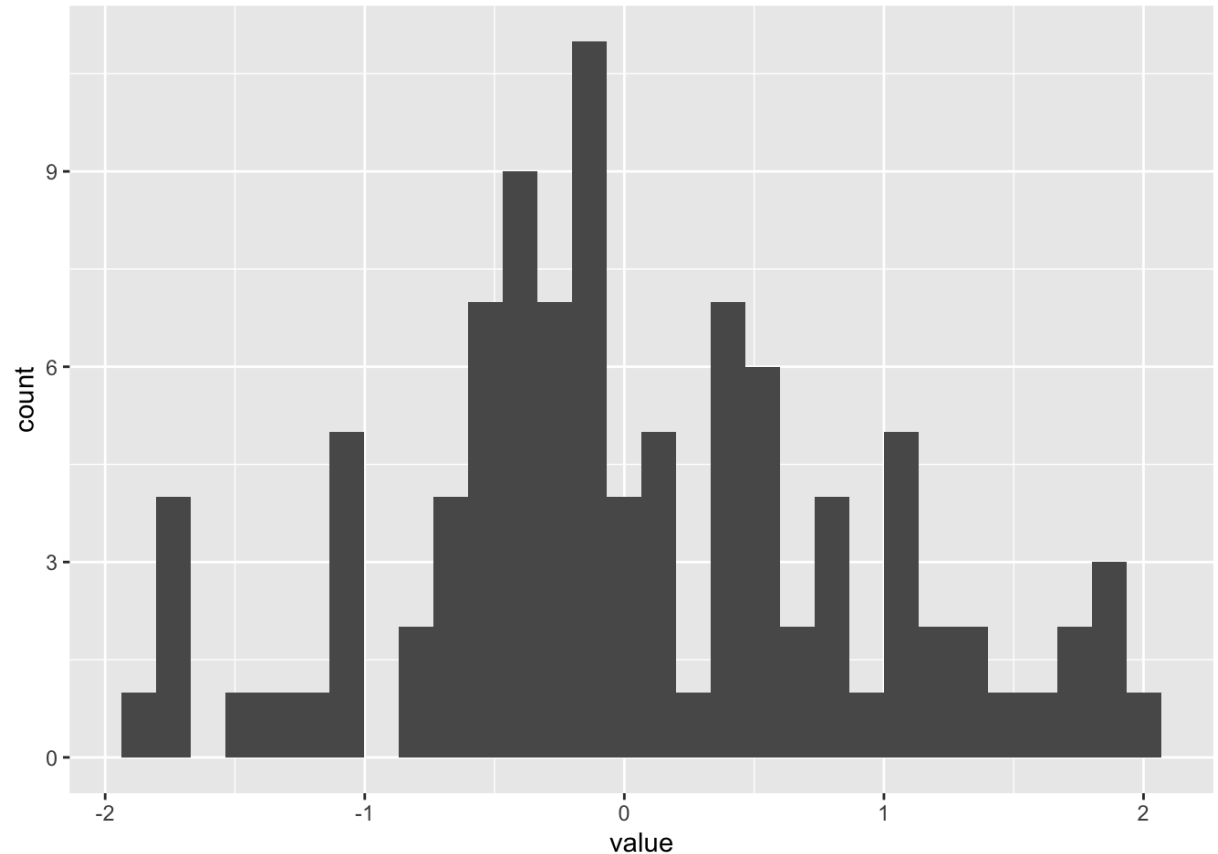
- Show **counts** or **averages** for categories.
- Example: Average exam score by class section.
-  *Good for:* Categorical data (like gender, major, condition).



Histograms

`geom_histogram()`

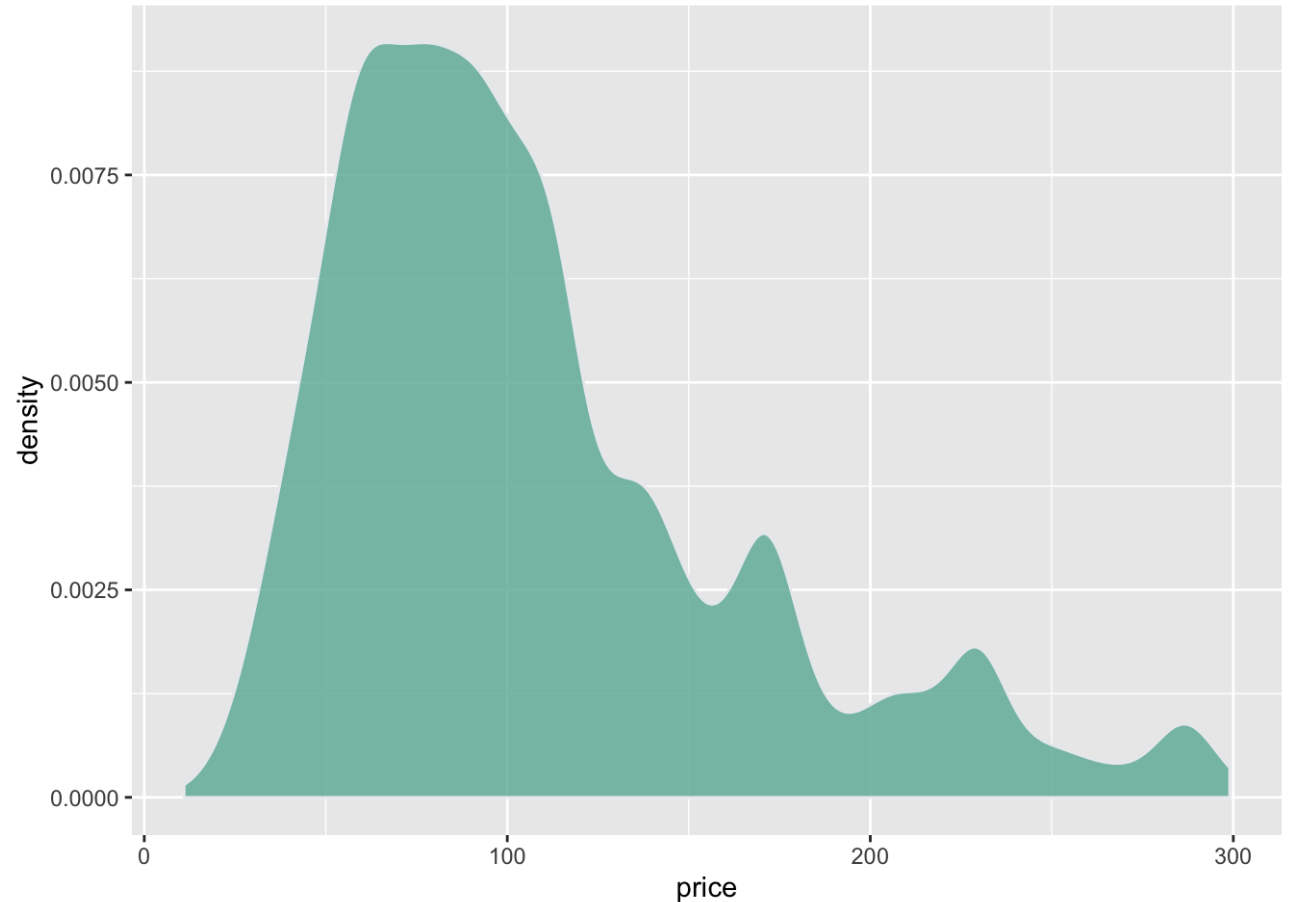
- Show **how many values** fall into each range.
- Example: Distribution of reaction times.
-  *Good for:* Seeing shape, center, and spread of one numeric variable.



Density Plots

`geom_density()`

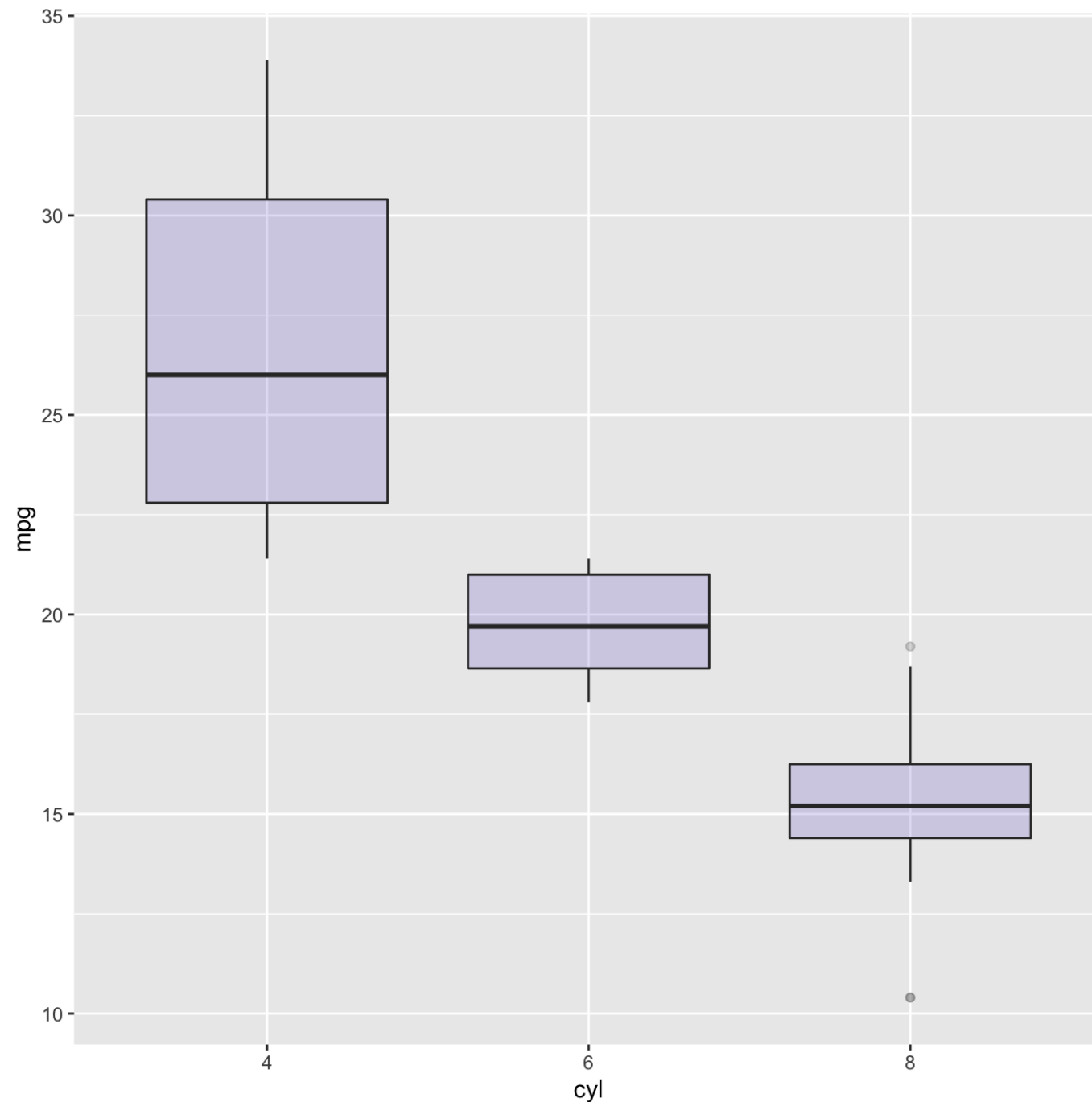
- A **smoothed version** of a histogram.
- Example: Comparing exam score distributions for two groups.
-  *Good for:* Comparing distributions and spotting skew.



Boxplots

geom_boxplot()

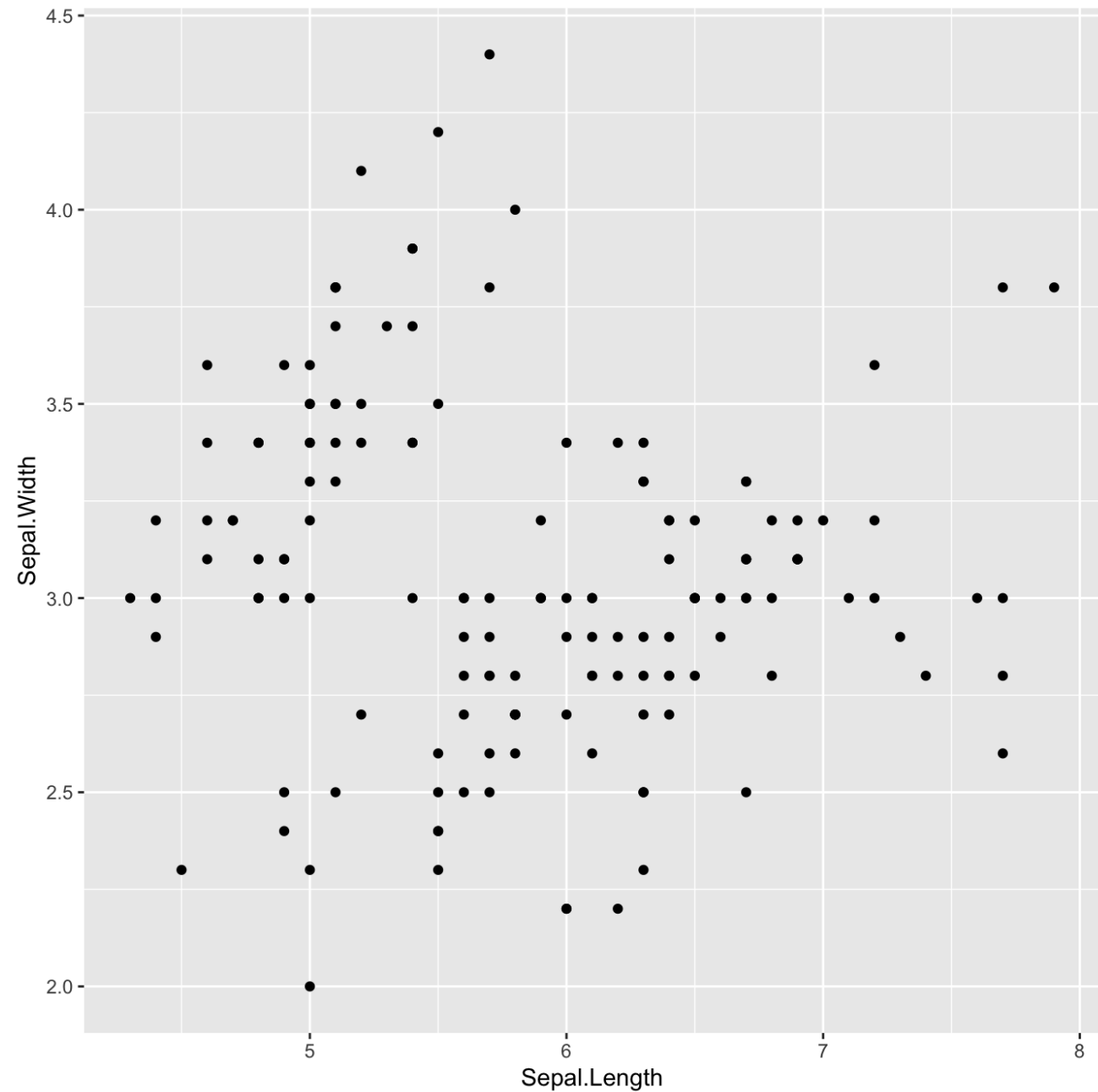
- Show **median**, **quartiles**, and **outliers**.
- Example: Scores by treatment group.
-  *Good for:* Comparing groups with numeric data.



Scatterplots

geom_point()

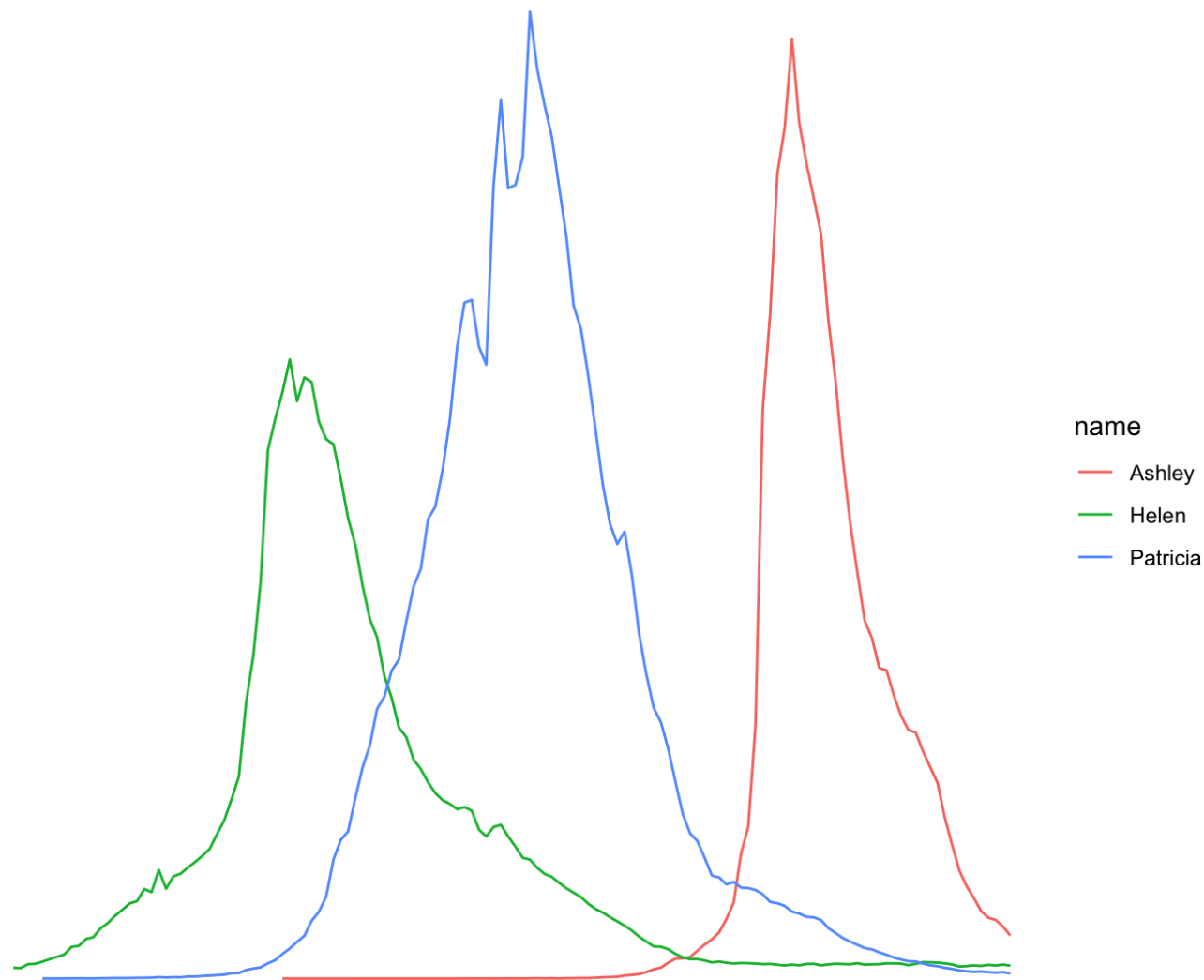
- Show **relationship between two numeric variables**.
- Example: Study hours vs. exam score.
-  *Good for:* Seeing patterns, clusters, and correlations.



Line Plots

geom_line()

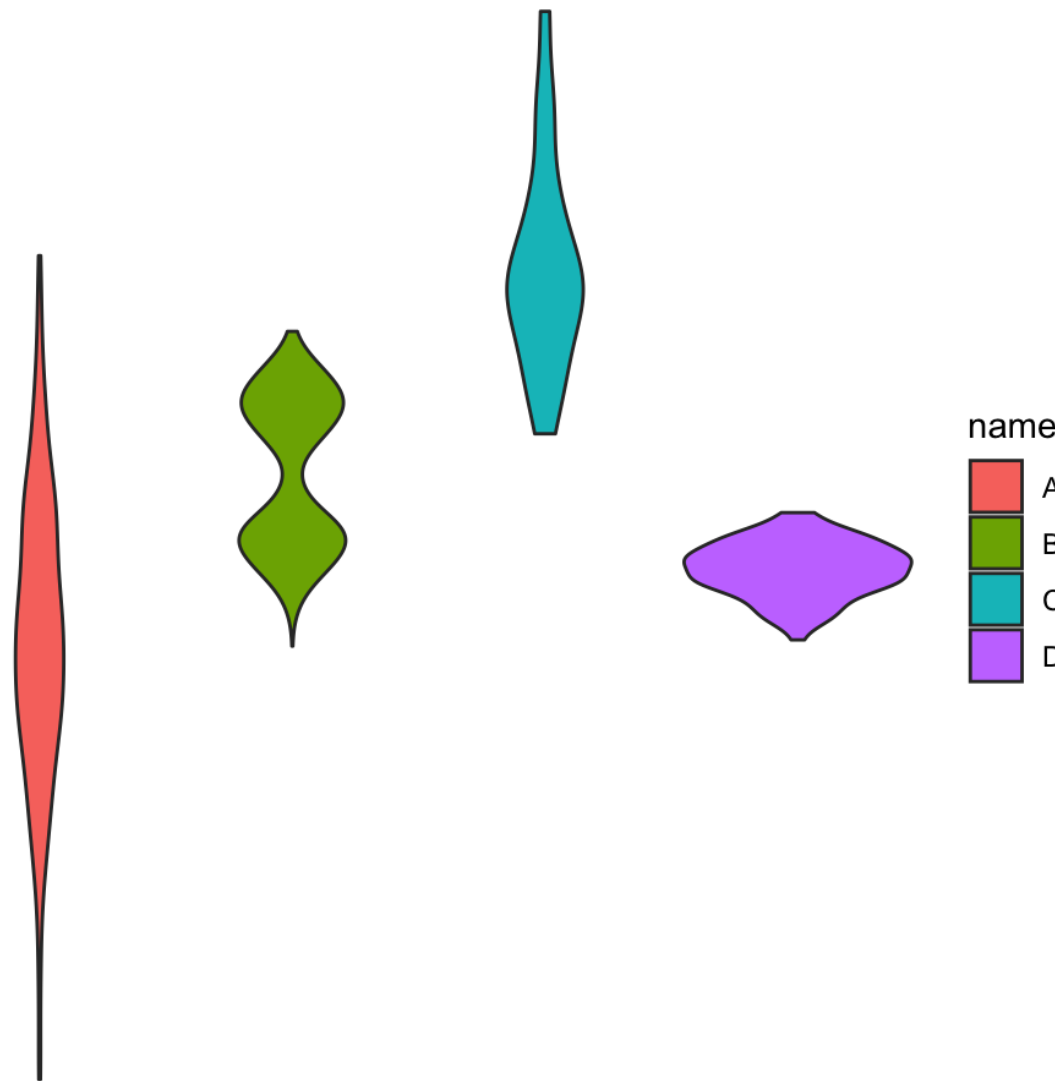
- Show **trends over time** or ordered categories.
- Example: Stress levels across weeks of the semester.
-  *Good for:* Time series or repeated measures.



Violin Plots

geom_violin()

- Combine **boxplot** and **density** info in one shape.
- Example: Grade distributions by course section.
-  *Good for:* Showing spread *and* shape at the same time.



Data (data)

FUNDAMENTALS

- Understand the structure of data frames (data.frame, tibble).
- Know how to pass data using data = your_dataframe.
- Ensure variables are correctly formatted (e.g., factors for categorical variables).
- `ggplot(data = mtcars)`

APPLICATIONS

- Filter or preprocess data using packages like dplyr before plotting.
- Use data within layers to plot subsets or different datasets on the same graph.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  geom_smooth(data = subset(mtcars, cyl == 4), method = "lm")
```

Aesthetics (aes())

- Understand basic mappings (x, y, color, size, shape).
- Realize that aesthetic mappings define how data is visually represented.

```
aes(x = wt, y = mpg, color = factor(cyl))
```

- Dynamically map aesthetics based on conditional statements or derived variables.
- Use aesthetic inheritance – define global mappings in `ggplot()` and override in layers.

```
ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +  
  geom_point() +  
  geom_smooth(aes(color = NULL), method = "lm") # Removes  
color mapping for this layer
```

Geometry (geom_*)

- Learn the most common geometries:
 - `geom_point()` – scatterplots
 - `geom_line()` – line charts
 - `geom_bar()` – bar charts
- Combine multiple geometries in one plot.
- Customize layers using arguments like `position`, `alpha`, and `stat`.
- Create custom geoms or modify existing ones using `ggproto`.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE)
```

Statistics (stat_*)

- Understand that geoms automatically apply default statistics (e.g., `geom_bar()` uses `stat_count()`).
- Use `geom_smooth()` for trend lines.
- Customize statistical transformations using `stat_*`.
- Create custom statistical summaries using `stat_summary()`.

```
geom_smooth(method = "lm")
```

```
geom_errorbar(stat = "summary", fun.data = mean_se)
```

Scales (scale_*)

- Learn basic scale adjustments:
 - `scale_x_continuous()` for continuous axes
 - `scale_color_manual()` for custom colors

`scale_x_continuous(limits = c(0, 100))`

- Use transformations (log, square root) for better data visualization.
`scale_y_log10()`
- Customize legends, breaks, and labels.

Facets (facet_*)

- Use `facet_wrap()` for one variable and `facet_grid()` for two variables.

```
facet_wrap(~cyl)
```

- Customize facet labels with `labeller`.
- Adjust scales independently for each facet (`scales = "free"`).

```
facet_wrap(~cyl, labeller = label_both)
```

Coordinates (coord_*)

- Use `coord_flip()` to swap x and y axes.
- Use `coord_cartesian()` to zoom without cropping data.
- Explore polar coordinates (`coord_polar()`) for circular plots.
- Use custom limits to focus on data without removing points.

`coord_flip()`

`coord_polar()`

Themes (theme())

- Use built-in themes for quick styling.

`theme_classic()`

`theme_minimal()`

- Fully customize the plot's appearance (fonts, margins, gridlines).
- Create custom themes for consistent styling across multiple plots.

```
theme(  
  axis.title = element_text(size = 14, face = "bold"),  
  panel.grid.major = element_blank()  
)
```